Project Title

Student Performance Analysis Using NumPy

1.Problem Statement

- Educational institutions often need to analyze student performance across different subjects to understand overall academic trends. Manual analysis of large datasets can be time-consuming and inefficient.
- Therefore, there is a need for a computational approach to analyze student marks efficiently and extract meaningful insights.
- In this project, we use NumPy, a powerful numerical computing library in Python, to analyze student marks and identify patterns in academic performance.

2.Objective

The main objectives of this project are:

- To understand how NumPy works with numerical datasets
- To perform statistical analysis on student marks
- To compute mean, median, and standard deviation
- To identify highest and lowest scores
- To analyze subject-wise performance
- To gain insights about student academic trends

3.Dataset Description

- The dataset contains information about 200 students and their marks in different subjects.
- All values are numerical, which makes this dataset suitable for NumPy numerical analysis.

4.Import Required Libraries

In [1]:
```
import numpy as np
import pandas as pd
```

Explanation:

- NumPy → used for numerical computations
- Pandas → used to load the dataset

5.Load the Dataset

In [4]:
```
df = pd.read_csv("student_marks.csv")

df.head()
```

Out[4]:

	Student_ID	Math	Physics	Chemistry	English	Computer_Science	Attendance_Perce
0	1	86	77	41	87	82	
1	2	49	32	75	86	71	
2	3	95	49	31	62	70	
3	4	55	53	64	54	55	
4	5	58	83	37	76	55	

6.Convert Data into NumPy Array

- NumPy works with arrays, so we convert the dataset.

In [6]:
```
data = df.to_numpy()

data
```

Out[6]:
```
array([[  1,  86,  77, ...,  87,  82,  71],
       [  2,  49,  32, ...,  86,  71,  85],
       [  3,  95,  49, ...,  62,  70,  72],
       ...,
       [198,  66,  73, ...,  84,  72,  90],
       [199,  43,  58, ...,  45,  96,  65],
       [200,  53,  42, ...,  76,  75,  86]], shape=(200, 7))
```

7.Attributes of Numpy (Basic Functions)

In [ ]:
```
data.shape # retrun no rows and columns
```

Out[ ]:
```
(200, 7)
```

In [ ]:
```
data.ndim # return array dimension
```

Out[ ]:
```
2
```

In [ ]:
```
data.size # return total element present in array
```

Out[ ]:
```
1400
```

In [ ]:
```
data.dtype # return which type of ele present in array
```

Out[ ]: dtype('int64')

8.Extract Subject Marks

- We only need subject marks (ignore Student_ID).

In [11]:
```
marks = data[:,1:6]

marks
```

```
Out[11]: array([[86, 77, 41, 87, 82],
                [49, 32, 75, 86, 71],
                [95, 49, 31, 62, 70],
                [55, 53, 64, 54, 55],
                [58, 83, 37, 76, 55],
                [37, 62, 55, 60, 97],
                [56, 53, 63, 53, 53],
                [87, 65, 36, 41, 79],
                [36, 67, 97, 50, 60],
                [64, 54, 87, 90, 79],
                [72, 47, 58, 78, 80],
                [36, 95, 65, 77, 73],
                [98, 83, 50, 99, 58],
                [94, 64, 65, 73, 52],
                [55, 90, 39, 77, 80],
                [67, 70, 53, 89, 89],
                [92, 62, 93, 73, 86],
                [56, 97, 78, 57, 85],
                [83, 62, 65, 69, 73],
                [93, 43, 53, 54, 80],
                [76, 50, 52, 66, 55],
                [94, 77, 91, 90, 51],
                [49, 49, 66, 73, 69],
                [96, 37, 41, 91, 77],
                [96, 36, 84, 96, 60],
                [81, 96, 42, 77, 53],
                [96, 46, 52, 72, 64],
                [85, 62, 59, 63, 55],
                [89, 77, 46, 54, 98],
                [98, 88, 91, 99, 79],
                [37, 51, 42, 69, 87],
                [85, 59, 88, 94, 51],
                [41, 67, 48, 81, 64],
                [55, 80, 78, 90, 60],
                [73, 83, 41, 56, 57],
                [52, 37, 90, 44, 75],
                [38, 56, 48, 68, 94],
                [94, 56, 38, 43, 93],
                [48, 50, 57, 49, 54],
                [43, 59, 81, 95, 55],
                [87, 57, 45, 56, 75],
                [36, 93, 98, 49, 53],
                [94, 98, 41, 56, 68],
                [78, 90, 54, 99, 69],
                [42, 77, 81, 59, 82],
                [81, 48, 82, 63, 69],
                [69, 33, 52, 44, 98],
                [70, 64, 45, 73, 61],
                [84, 93, 86, 45, 96],
                [38, 78, 68, 92, 50],
                [36, 46, 82, 41, 75],
                [40, 73, 71, 52, 63],
                [88, 59, 87, 82, 87],
                [38, 75, 68, 82, 86],
                [88, 35, 43, 87, 60],
                [97, 66, 34, 50, 85],
                [52, 53, 64, 90, 62],
                [78, 75, 47, 86, 92],
                [68, 82, 38, 96, 52],
                [96, 89, 87, 62, 82],
```

```
    [48, 92, 46, 94, 55],
    [82, 61, 36, 55, 99],
    [49, 62, 75, 70, 59],
    [96, 96, 42, 50, 54],
    [74, 47, 69, 93, 72],
    [87, 54, 71, 55, 59],
    [58, 83, 38, 47, 93],
    [60, 87, 79, 43, 51],
    [94, 96, 56, 79, 62],
    [75, 75, 95, 43, 89],
    [63, 53, 34, 95, 51],
    [49, 61, 58, 64, 69],
    [79, 76, 66, 42, 50],
    [99, 52, 67, 71, 86],
    [43, 95, 37, 42, 58],
    [35, 56, 94, 66, 66],
    [42, 31, 46, 68, 58],
    [97, 46, 74, 71, 60],
    [45, 62, 33, 89, 64],
    [42, 38, 65, 97, 73],
    [69, 72, 99, 90, 87],
    [69, 77, 60, 58, 84],
    [67, 68, 48, 60, 79],
    [39, 71, 90, 44, 80],
    [75, 55, 83, 57, 98],
    [62, 79, 68, 67, 54],
    [41, 54, 48, 81, 93],
    [46, 53, 68, 61, 99],
    [68, 42, 96, 60, 63],
    [67, 89, 74, 45, 60],
    [82, 36, 42, 40, 58],
    [57, 86, 87, 44, 83],
    [96, 65, 49, 80, 61],
    [71, 74, 90, 51, 84],
    [78, 49, 68, 65, 84],
    [69, 94, 30, 85, 50],
    [99, 37, 32, 73, 89],
    [81, 45, 91, 88, 71],
    [37, 43, 92, 53, 78],
    [35, 44, 54, 65, 57],
    [39, 95, 85, 84, 60],
    [48, 61, 62, 66, 90],
    [61, 92, 67, 48, 86],
    [43, 80, 35, 65, 63],
    [49, 54, 87, 96, 79],
    [76, 87, 73, 86, 84],
    [85, 92, 74, 96, 70],
    [97, 91, 61, 61, 86],
    [86, 51, 74, 86, 54],
    [38, 87, 90, 95, 68],
    [57, 87, 76, 69, 63],
    [49, 78, 50, 82, 75],
    [77, 81, 65, 87, 53],
    [63, 71, 48, 56, 74],
    [70, 99, 49, 65, 94],
    [47, 44, 86, 75, 91],
    [66, 83, 47, 40, 74],
    [93, 89, 76, 47, 67],
    [62, 37, 78, 88, 89],
    [76, 82, 43, 74, 57],
```

```
[79, 89, 44, 91, 88],
[96, 34, 60, 54, 89],
[91, 97, 30, 86, 63],
[40, 35, 83, 95, 81],
[62, 76, 32, 61, 87],
[62, 84, 45, 53, 82],
[78, 69, 86, 65, 72],
[64, 81, 41, 67, 64],
[96, 45, 45, 62, 82],
[96, 42, 53, 53, 74],
[35, 59, 57, 63, 66],
[61, 48, 37, 41, 82],
[96, 46, 65, 84, 96],
[37, 92, 37, 65, 51],
[61, 48, 87, 53, 63],
[43, 87, 89, 98, 89],
[96, 84, 79, 95, 89],
[71, 91, 57, 90, 88],
[85, 52, 70, 46, 55],
[78, 38, 93, 42, 55],
[58, 41, 56, 86, 52],
[93, 30, 92, 62, 56],
[66, 87, 46, 85, 57],
[86, 30, 62, 82, 91],
[96, 63, 58, 86, 64],
[92, 77, 42, 99, 96],
[86, 30, 75, 84, 78],
[46, 45, 64, 57, 82],
[73, 90, 35, 97, 79],
[36, 93, 98, 77, 88],
[37, 92, 76, 74, 76],
[90, 98, 54, 90, 85],
[93, 51, 95, 54, 78],
[36, 96, 39, 64, 87],
[36, 55, 85, 94, 82],
[88, 45, 59, 76, 86],
[35, 80, 34, 67, 76],
[53, 86, 62, 49, 82],
[36, 58, 94, 78, 53],
[87, 98, 47, 96, 71],
[78, 76, 78, 56, 51],
[66, 91, 40, 78, 59],
[66, 98, 55, 61, 54],
[89, 45, 92, 65, 59],
[90, 77, 88, 83, 82],
[51, 68, 56, 64, 87],
[72, 62, 78, 56, 62],
[58, 52, 62, 52, 80],
[45, 39, 30, 59, 96],
[50, 98, 50, 64, 85],
[93, 63, 84, 43, 94],
[37, 81, 35, 49, 73],
[54, 39, 98, 42, 64],
[93, 48, 34, 80, 78],
[70, 87, 32, 84, 57],
[53, 30, 82, 57, 54],
[53, 98, 52, 86, 78],
[54, 33, 82, 75, 96],
[86, 45, 66, 86, 53],
[67, 53, 46, 61, 61],
```

```
       [74, 31, 30, 73, 94],
       [73, 61, 80, 86, 51],
       [35, 53, 74, 47, 76],
       [45, 41, 33, 79, 80],
       [91, 79, 91, 88, 85],
       [84, 64, 94, 83, 85],
       [57, 62, 61, 58, 75],
       [65, 62, 63, 81, 92],
       [76, 90, 68, 80, 76],
       [41, 80, 55, 76, 54],
       [50, 72, 63, 45, 69],
       [94, 41, 83, 91, 60],
       [36, 96, 32, 65, 59],
       [35, 94, 79, 73, 89],
       [82, 62, 41, 98, 87],
       [46, 69, 94, 95, 55],
       [71, 72, 83, 98, 57],
       [66, 73, 34, 84, 72],
       [43, 58, 86, 45, 96],
       [53, 42, 46, 76, 75]])
```

9.Calculate Average Marks of Each Subject

In [ ]:
```
average_marks = np.mean(marks, axis=0)

average_marks
```

Out[ ]:
```
array([66.77 , 66.16 , 62.91 , 70.245, 72.675])
```

Explanation:

- axis = 0 → column wise calculation
- This gives the average marks of each subject.

10. Find Highest Marks in Each Subject

In [13]:
```
highest_marks = np.max(marks, axis=0)

highest_marks
```

Out[13]:
```
array([99, 99, 99, 99, 99])
```

- This shows the top score in every subject.

11. Find Lowest Marks

In [14]:
```
lowest_marks = np.min(marks, axis=0)

lowest_marks
```

Out[14]:
```
array([35, 30, 30, 40, 50])
```

- This identifies minimum marks in each subject.

12. Calculate Standard Deviation

In [15]:
```
std_dev = np.std(marks, axis=0)

std_dev
```

Out[15]:
```
array([20.49602644, 20.00685882, 19.9747816 , 17.23702338, 14.33524939])
```

- Standard deviation helps understand score variation among students.

13. Find Overall Average of Each Student

In [16]:
```
student_average = np.mean(marks, axis=1)

student_average[:10]
```

Out[16]:
```
array([74.6, 62.6, 61.4, 56.2, 61.8, 62.2, 55.6, 61.6, 62. , 74.8])
```

Axis explanation:

- axis = 1 → row wise calculation

14. Identify Top Performing Students

In [17]:
```
top_students = np.where(student_average > 85)

top_students
```

Out[17]:
```
(array([ 29, 136, 184]),)
```

15. Identify Students Needing Improvement

In [18]:
```
low_students = np.where(student_average < 40)

low_students
```

Out[18]:
```
(array([], dtype=int64),)
```

16. Total Marks of Each Student

- This calculates the sum of marks across all subjects for each student.

In [19]:
```
total_marks = np.sum(marks, axis=1)

total_marks[:10]
```

Out[19]:
```
array([373, 313, 307, 281, 309, 311, 278, 308, 310, 374])
```

Explanation:

- axis=1 → row-wise calculation
- Gives total marks of each student.
- Use case: helps identify overall academic performance.

17. Overall Class Average

- Calculate the average marks of the entire class.

In [20]:
```
class_average = np.mean(marks)

class_average
```

Out[20]:
```
np.float64(67.752)
```

- This gives the overall mean score of all students across all subjects.

18. Identify Topper of the Class

- Find the student with highest total marks.

In [21]:
```
topper_index = np.argmax(total_marks)

topper_index
```

Out[21]:
```
np.int64(29)
```

Explanation:

- argmax() returns the index of maximum value.

- You can also display the topper:

In [22]:
```
df.iloc[topper_index]
```

Out[22]:
```
Student_ID               30
Math                     98
Physics                  88
Chemistry                91
English                  99
Computer_Science         79
Attendance_Percentage    60
Name: 29, dtype: int64
```

19.Identify Weakest Student

- Find student with lowest total marks.

In [23]:
```
weak_student = np.argmin(total_marks)

df.iloc[weak_student]
```

Out[23]:
```
Student_ID               77
Math                     42
Physics                  31
Chemistry                46
English                  68
Computer_Science         58
Attendance_Percentage    88
Name: 76, dtype: int64
```

20.Students Scoring Above Class Average

- Find students performing better than the class average.

In [24]:
```
above_average_students = np.where(student_average > class_average)

above_average_students
```

Out[24]:
```
(array([  0,   9,  11,  12,  13,  14,  15,  16,  17,  18,  21,  23,  24,
         25,  28,  29,  31,  33,  42,  43,  44,  45,  48,  52,  53,  57,
         59,  64,  68,  69,  73,  77,  80,  81,  84,  91,  92,  93,  94,
         97, 100, 102, 104, 105, 106, 107, 108, 109, 110, 112, 114, 115,
        117, 118, 120, 122, 126, 132, 135, 136, 137, 142, 143, 144, 145,
        146, 148, 149, 150, 151, 152, 154, 155, 159, 160, 163, 164, 169,
        170, 176, 177, 181, 184, 185, 187, 188, 191, 193, 194, 195, 196]),)
```

Explanation:

- Returns indices of students performing above average.

21. Subject-wise Highest Score Index

- Find which student scored highest in each subject.

In [28]:
```
subject_toppers = np.argmax(marks, axis=0)

subject_toppers
```

Out[28]:
```
array([ 73, 114,  80,  12,  61])
```

Explanation:

- Each value represents student index who scored highest in that subject.

22. Sort Students by Average Marks

- Sort students by academic performance.

In [29]:
```
sorted_students = np.sort(student_average)

sorted_students[:10]
```

Out[29]: array([49. , 51. , 51.6, 51.6, 53.8, 53.8, 55. , 55. , 55.2, 55.6])

23. Count Students Scoring Above 80

- Find number of high-performing students.

In [30]:
```
high_performers = np.sum(student_average > 80)

high_performers
```

Out[30]: np.int64(16)

24. Insights

From the analysis, we can observe:

- Some subjects have higher average marks, indicating better student performance.
- Standard deviation shows variation in student scores.
- A group of students consistently scores above 85, indicating high academic performance.
- The class topper achieved the highest overall marks across all subjects.
- A number of students are performing above the class average, indicating good academic performance.
- A small group of students requires academic improvement and support.

25. Recommendations

Based on the analysis:

- Teachers should provide extra support to low-performing students.
- Subjects with lower average marks may require improved teaching strategies.
- High-performing students could be encouraged with advanced learning opportunities.
- Attendance should be monitored since it may impact academic performance.

26.Conclusion

This project demonstrated how NumPy can be used for numerical data analysis. Using NumPy functions such as mean, min, max, and standard deviation, we efficiently analyzed student performance across multiple subjects.

The analysis helped identify academic trends, top performers, and students needing improvement. NumPy proves to be a powerful tool for handling large numerical datasets and performing fast statistical computations.